

TASK 1

Person类:

```
public class Person {
    private String name;
    private int age;
    private int sex;

    public Person(){    //空参构造方法
    }

    public Person(String name, int age, int sex) {    //全参构造方法，赋值用到的(为什么sex是个整数
    (??))
        this.name = name;
        this.age = age;
        this.sex = sex;
    }
    private void eat() {
        System.out.println(name + "正在吃东西");
    }

    private void sleep() {

    }

    private void dadoudou() {

    }
}
```

测试类:

```
public class Test {
    public static void main(String[] args) {
        Person p = new Person("微光娘",19,0);    //由于有构造方法，创建对象时直接填上参数即可
    }
}
```

• 构造方法是否用到了this？在这里有什么用？

我这里用到了this，用来表示后面的变量名是在方法外面的**成员变量**，避免方法内的局部变量的重名导致的问题

这里带有this的name/age/sex代表成员变量，没有this的按就近原则查找，优先指方法内的局部变量

• 类和对象的关系

总体来说，类是对象的一个模板、定义，而对象是类的具体存在的实例。

一般来说，一个类对应多个对象，类定义了一系列如身高、吃饭睡觉这样的数据和方法，它并不是实际存在占用内存的，它定义了对象应该含有的特点。

而对象就是类的具体体现，它是一个实例，真正具有了类所定义的这些数据和方法

TASK 2

Person类：

```
public class Person {
    private String name;
    private int age;
    private int sex;

    private static int count; //增加了一个静态变量，统计新建的对象数

    public Person(){ //空参构造方法
        count++; //如果创建方法没有写参数那就是这里++
    }

    public Person(String name, int age, int sex) { //全参构造方法，赋值用到的
        this.name = name;
        this.age = age;
        this.sex = sex;
        count++; //如果填写了全参那就是这里
    }
    //这里我把下面的几个方法的访问修饰符都改成了public，不然在类的外面都没法调用，没有意义
    public void eat() {
        System.out.println(name + "正在吃东西");
    }

    public void sleep() {
        System.out.println(name + "正在睡觉");
    }

    public void dadoudou() {
        System.out.println(name + "正在打豆豆");
    }
    //我这里新添加了一个getCount静态方法用来在其他类获取上面count变量的值
    public static int getCount() {
        return count;
    }
}
```

因为访问修饰符都是齐了的，题目说的补齐我不太理解，所以我就进行一些更改吧ww

• 在不同位置访问

这里三个字段是private，三个方法是public

- 当前类内部：
字段和方法都可以访问
- 同一个包之下的其他类：
字段没法访问，方法可以
- 不同包下的类：
同上，字段不行，方法可以

• 实验一下

我建了两个Test类，分别在同一个包下和另外一个包。我在这两个类和Person类里写了三个psvm，然后创建对象名为p

发现除了本类，带private的 `p.name` 直接写都是直接变红，不过那三个public方法全部都可以调用

还有一个geCount静态方法，我在Person类里加了一个show()方法，然后直接在本类的main里写show()，输出了这样一段error:

```
Non-static method 'show()' cannot be referenced from a static context
```

这个意思应该就是非静态方法show()不能被静态方法省略对象引用直接调用吧，改成p.show()就行了

再然后把show前面加上了一个static，在main里不创建对象直接写Person.show()发现是可以的

目前简单得出结论：private权限只有在当前的类才能访问，public任意位置都可以；静态方法里无法不加对象直接调用非静态方法，只能创建对象用对象名引用，且静态方法可以直接使用类名来调用

• public、private、protected、默认访问权限分别代表什么

- **public**
任何地方都可以访问
- **private**
只有当前类内部可以访问
- **protected**
同一个包下的其他类或者是对应的子类可以访问
- **默认权限**
同一个包下的类之间可以访问

• 再添加一个静态方法（见上面代码块）

- 为什么静态方法可以直接通过类名调用？

因为**静态方法属于类**，在对应的类加载好了之后，这个静态方法就已经存在，且它不需要依赖对象里的数据，所以可以直接类名调用

- 为什么实例方法通常需要先创建对象？

因为实例方法属于实例，也就是对象，它一般对对象里面的变量进行操作，要先创建一个对象才有可以调用的数据。

TASK 3

父类：

```
public class Animal {
    private String name;
    private int id;

    public Animal(String myName, int myId) {
        name = myName;
        id = myId;
    }

    public void eat() {
        System.out.println(name + "正在吃");
    }

    public void sleep() {
        System.out.println(name + "正在睡");
    }

    public void introduction() {
        System.out.println("大家好！我是" + id + "号" + name + "。");
    }
}
```

Penguin类：

```
package CLASSTEST;

public class Penguin extends Animal {
    public Penguin(String myName, int myId){
        super(myName, myId); //调用父类构造方法
    }
    //重写方法，体现差异：吃的具体细节不同
    @Override
    public void eat(){
        System.out.println("正在吃小鱼");
    }
}
```

Mouse类：

```
package CLASSTEST;

public class Mouse extends Animal{
    public Mouse(String myName, int myId) {
        super(myName, myId); //调用父类构造方法
    }
}
```

- **重写父类方法**：在Penguin类里重写了eat，体现细节不同，加上@Override让编译器帮我核对重写方法是否正确

• 为什么Java 不支持类的多继承？

Java中，一般把多个子类的共同数据和方法提出来写在父类中，这样子类就可以从父类直接继承、复用，节省代码。

一个子类对应多个父类（也就是类的多继承），如果两个父类没有相同的数据，那还好；但倘若两个父类**都有同样名称的数据和方法**，就会出现冲突。

比如，一个Penguin子类，它有Animal和SeaAnimal两个父类，还有个间接父类“爷爷类”Creature

我们假设Animal和SeaAnimal两个类都有name变量，**且内容不一**，那么我在Penguin里想要调用父类的name，应该调用谁的？这产生了一个普通的继承冲突

类似地，Creature里面有一个sleep方法而两个父类里都对这个sleep方法进行了重写，内容还不相同

那么此时我在测试类里创建Penguin的对象，调用sleep方法，应该如何执行？这就是产生了一个菱形继承冲突的典型

以此为例，为了避免这些一系列的列危害，所以Java不支持类的多继承

TASK 4

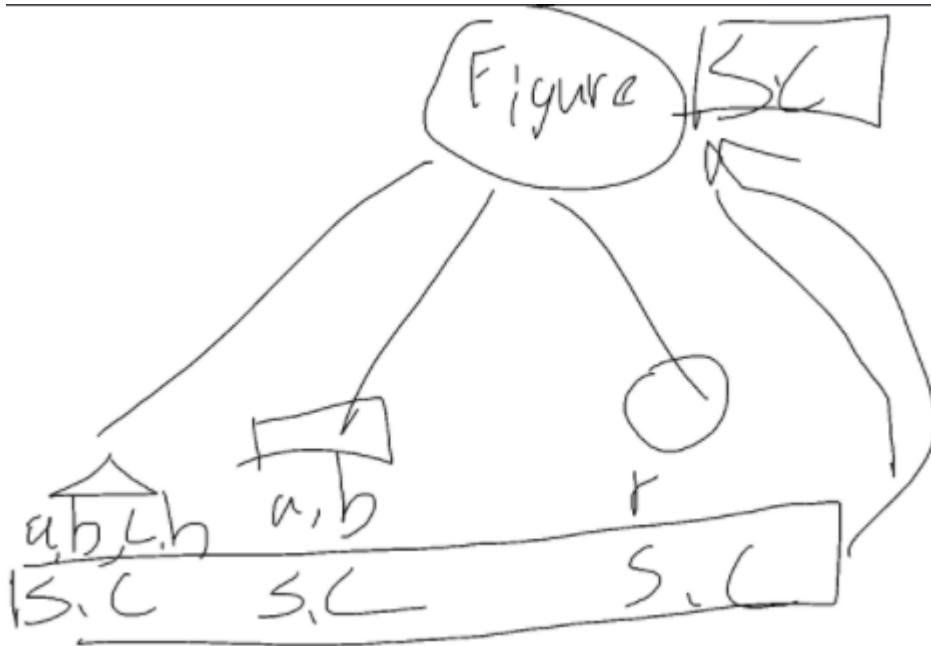
我这里使用抽象类方案写

为了尽快熟悉写面向对象，我这里把Circle手打了，后面发现类太多，变量也太多（尤其是三角形）

于是我其他的类getset方法全部用了alt+insert，构造方法也Tab了(

开始我画了个继承关系的树状草图，把父类写了一些成员变量，但是其他的变量过于“个性化”，比如半径、高.....

最后才发现思路有问题，实际上只能提出两个计算的方法来 ⚠ 艺术程度过高 ⚠



父类 (Figure) :

```
package figure;
//父类Figure，包含抽象方法getArea和getPerimeter，无其他属性，所以省略构造方法了
public abstract class Figure {
    public abstract double getArea();
    public abstract double getPerimeter();
}
```

Circle类:

```
package figure;
//圆的类，继承自Figure类，主要要素有半径和算面积、算周长两个方法
public class Circle extends Figure {
    private double r;//半径

    //父类是抽象类，这里重写算面积和算周长两个方法
    @Override
```

```

public double getArea(){
    return Math.PI*r*r;
}
@Override
public double getPerimeter(){
    return 2*Math.PI*r;
}
//带参构造方法，初始化半径
public Circle(double r) {
    this.r = r;
}
//空参构造方法
public Circle(){

}
//get和set方法
public double getR(){
    return r;
}

public void setR(double r){
    this.r = r;
}
}

```

矩形类：

```

package figure;

public class Rectangle extends Figure{
    private double length;
    private double width;
    //空参构造
    public Rectangle(){

    }
    //全参构造
    public Rectangle(double length, double width){
        this.length = length;
        this.width = width;
    }
    //get和set方法（依旧一键生成
    public double getLength() {
        return length;
    }

    public void setLength(double length) {
        this.length = length;
    }
}

```

```

    public double getWidth() {
        return width;
    }

    public void setWidth(double width) {
        this.width = width;
    }
    //开始重写方法
    @Override
    public double getArea(){
        return length*width;
    }
    @Override
    public double getPerimeter(){
        return 2*(length+width);
    }
}

```

三角形类:

```

package figure;

public class Triangle extends Figure {
    private double a;//底
    private double h;//高
    private double b;//腰1
    private double c;//腰2

    //全参构造方法
    public Triangle(double a, double h, double b, double c) {
        this.a = a;
        this.h = h;
        this.b = b;
        this.c = c;
    }
    //空参构造方法
    public Triangle() {
    }

    //get和set方法（一键生成真好用吧）
    public double getA() {
        return a;
    }
    public void setA(double a){
        this.a = a;
    }
    public double getH(){
        return h;
    }
    public void setH(double h){
    }
}

```



```

        this.h = h;
    }
    public double getB(){
        return b;
    }
    public void setB(double b){
        this.b = b;
    }
    public double getC(){
        return c;
    }
    public void setC(double c) {
        this.c = c;
    }

    //重写算面积周长的方法
    @Override
    public double getArea(){
        return a*h/2;
    }
    @Override
    public double getPerimeter(){
        return a+b+c;
    }
}

```

关于接口写法为什么不太恰当&抽象和接口到底适用在哪？：

由于这几个几何图形只有共同的方法(计算周长面积)，没有共同的属性提出共同的成员变量，所以用抽象父类的写法不需要手写带参的构造方法

这样一来，**接口写法和抽象类写法的区别在这里大大减小了**，小到仅仅是把class换成interface，把extends换成了implements

不过，这里的“各个几何图形”和“总几何图形”就是一个分类的关系，

况且，抽象类的主要用法在于①提取出各个子类共同的属性和方法以提高代码的复用性；②强制子类重写父类的抽象方法以提高代码的统一性。

而接口主要用于**“附加”上**，例如，可能部分子类有共同的方法，而再添加一个抽象父类太麻烦甚至让继承结构更不清晰，接口才派的上用场

这里每一个几何图形都有共同的方法，**使用接口写法来完成确实不太恰当**

• 下面利用集合存放对象然后遍历，循环调用周长面积方法

测试类：

这里我写的时候也遇到了一点小问题

我在写到遍历的时候，最开始的思路就是像我现在写的这个fori一样，用正常for循环，像c语言一样的遍历

可是，IDEA直接就给我弹了这个写法，循环条件我都看不懂？？

```
for (Figure f : list) {  
    System.out.println("面积: " + f.getArea());  
    System.out.println("周长: " + f.getPerimeter());  
}
```

搜了下这个叫**增强for循环**，语法模板是 `for(元素类型 临时变量名 : 集合/数组)`

执行流程大概是这样：

编译器拿到我给的list集合之后，调用list.iterator()生成了一个迭代器对象，它的初始位置在第一个元素前面，用来记录遍历的位置

然后再判断下一位还有没有元素，如果有，返回true，循环继续，取出下一位次的对象给到临时的变量f；如果没有，返回false，循环退出

遍历ArrayList集合时，它的底层是while循环

这段代码是我第一次实际体会到多态的优势，只用一个父类类型的引用变量f，就可以循环用三个图形的对象，如果我不知道的话可能还在一个一个的new呢（其实我第一次写这个测试类的时候就是这样一个一个来的

```
package figure;  
import java.util.ArrayList;  
public class Test {  
    public static void main(String[] args) {  
  
        //先依次把我需要的图形对象add进集合里  
        ArrayList<Figure> list =new ArrayList<>();  
        list.add(new Rectangle(10,5));  
        list.add(new Triangle(10,5,6,8));  
        list.add(new Circle(5));  
        //fori遍历集合，轮流把对象给到引用变量f，去循环调用面积和周长方法  
        for (int i = 0; i < list.size(); i++) {  
            Figure f = list.get(i);  
            System.out.println(f.getArea());  
            System.out.println(f.getPerimeter());  
        }  
    }  
}
```

TASK 5

- **public、private 等关键字在封装中的作用**

封装相关的权限修饰关键字主要有 `public`、`private` 和 `protected`，这里分一点详略展开

- **private**

`private`关键字可以说是封装的核心(它的意思也和封装很像)，它可以把权限改成私有，不让类本类以外的类对其进行访问。

有什么意义呢？

最主要的就是可以**避免数据的污染**。拿之前圆那个类举例子，我当时默认了半径`r`是一个有意义的数($r \geq 0$)。可是，万一我给了一个-1呢？给了一个0呢？这只是一个小型练习程序，导致的结果无非就是输出结果怪怪的，但一旦到了大型项目，这一类非法的数据就会**导致一定的危害**。如果我把数据使用`private`给私有了，**外面只能用getset方法来干涉数据**，这时，我在`set`方法里写一个输入数据的规则，不就能避免数据的污染了吗？

当然，还有对数据进行保密等等作用，这里不详细叙述

- **public**

`public`关键字作用地位个人认为等同于`private`。在把数据封装起来之后，外部需要`getset`方法来访问数据；在其他类需要继承父类方法时，抽象方法可以规定一个标准并强制重写.....

`public`关键字的主要作用在于在本类开辟一个可以对外暴露的接口(不是`interface`)，比如构造方法、`getset`方法、抽象方法.....这些在一个类里对于类与类之间的关联是非常必要的，此时`public`权限的公共优势就体现了出来

- **protected**

`protected`关键字我还没有尝试使用过，不过主要作用大概就是类似于`public`，不过是把范围控制在包内或者是子类，属于开辟一个范围更小的其他类的入口吧

当然还有不写关键字的默认权限，不过这应该不太好，不利于权限的清晰控制

BankAccount类：

看前须知：题目对于`validatePassword`和`validateAmount`两个方法仅仅要求修改可见性，但是这里许多需要核对密码的部分都需要调用这个方法。所以我在`validatePassword`那里加了一句修改注释→`return this.password.equals(inputPassword)`，不然咋样都是`true`

```
package bank;

public class BankAccount {
    // TODO: 修改属性的可见性
    //由于这里的信息保密性较强，且不可随意改动传入非法参数（比如余额），因此将权限全部都设置为
    private
        private String accountNumber;    //卡号
        private String accountHolder;    //账户持有人
        private double balance;          //余额
        private String password;          //密码
    //正常写带参构造方法，构造方法当然用public
```

```

    public BankAccount(String accountNumber, String accountHolder, double
initialBalance, String password) {
        // TODO
        this.accountHolder=accountHolder;
        this.accountNumber=accountNumber;
        //由于金额不能为负数或0，这里加一个条件判断，如果填了非法信息自动归零以免出问题
        if(initialBalance>0)
            this.balance=initialBalance;
        else
            this.balance=0;
        this.password=password;
    }
    //题干没有，但是还是补上一个空的构造方法吧(
    public BankAccount() {
    }

    //这是个存钱的方法，由于外部还是需要调用的，这里使用public
    public void deposit(double amount, String inputPassword) {
        // TODO
        if (validatePassword(inputPassword))
            this.balance+=amount;    //余额增加
        else
            System.out.println("密码错误");
    }

    //取款，同上一个方法，这些都是外部需要的方法，public
    public boolean withdraw(double amount, String inputPassword) {
        // TODO
        if(validatePassword(inputPassword)) {    //密码正确与否，调用校验方法
            if (amount > balance) {
                return false;    //余额不足，取款失败
            }else{
                this.balance -= amount;    //余额减少
                return true;
            }
        }else{
            return false;    //密码错误，取款失败
        }
    }

    //转账
    public boolean transfer(BankAccount recipient, double amount, String
inputPassword) {
        // TODO
        if(validatePassword(inputPassword)) {    //密码正确与否，调用下面的校验方法
            if (amount>balance){
                return false;    //余额不足无法转账
            }else{
                this.balance-=amount;
                recipient.deposit(amount, inputPassword);//收款，也就是收款方进行了存
款

                return true;    //转账成功
            }
        }else{
            return false;    //密码错误，转账失败
        }
    }

```

```

    }

}

//获取账户余额的信息，返回余额，get方法当然public
public double getBalance(String inputPassword) {
    // TODO
    if(validatePassword(inputPassword)) {
        return this.balance;
    }else{
        return -1;
    }
}

//获取账户的信息，属于get方法，这里返回卡号、持有人、余额三个信息（密码就不来了）
public String getAccountInfo(String inputPassword) {
    // TODO
    // 看信息也要密码
    if(validatePassword(inputPassword)) {
        //用substring把卡号后四位拿出来，方便后面保密
        String end=accountNumber.substring(accountNumber.length()-4);
        return "卡号:" + "****" +end+ "\n账户持有人: " + accountHolder + "\n余额: " + balance;
    }else{
        return "密码错误";
    }
}

// 只需修改可见性
//可见性修改为private,这个只需要在类内访问用来验证密码
private boolean validatePassword(String inputPassword) {
    return true;    //这里确实可以改成: return
this.password.equals(inputPassword);
}

// 只需修改可见性
//注释见上一个方法
private boolean validateAmount(double amount) {
    return true;
}
}

```

设立的规则：

1. 金额不能是负数或零（构造方法里加了个判断）
2. 取款、转账需要密码
3. 余额不能被外部代码随意修改（private修饰一下，不写set方法，然后其他余额的地方加上一个密码校验）
4. 不展示敏感信息（把卡号只露出后四位不过星号个数没法控制，算是个缺陷吧，能力不足ww）
5. 查看信息和余额、存款也需要密码

这里我把Person类的代码和运行图片放在下面看看部分方法的效果

关于我遇到的一点小问题和代码里面的小问题

1. BankAccount类里面有一个不符合现实状况的小bug其实

由于题目不方便增加新的方法，我直接把存钱的方法用在转钱的收款方那里了
但是这样万一收款方的密码不对还转不了钱，现实里转钱当然不用收款方密码（
这一点是代码的一个小缺陷，当然更多是我懒了（

2. Person类调用方法遇到的问题

刚开始我设置了余额100，转账1314，结果发现转了之后余额变成了2728（？）
后面发现罪魁祸首了，算是低级小错误吧（

我最开始写的转账

```
b.transfer(bb, 1314.0, "iloveweiguangniang")
if(b.transfer(bb, 1314.0, "iloveweiguangniang")) {
    System.out.println("转账成功! ");
}
else {
    System.out.println("转账失败! ");
}
```

我一开始单线程了，认为if里面的语句单纯就是个判断作用，后面想想不对，既然我都拿方法执行之后的return值来判断了，那方法不就已经被执行了吗？后面删除第一行就行了

Person类：

```
package bank;

public class Person {
    public static void main(String[] args) {
        BankAccount b=new BankAccount("1145141919810", "Ellery",
1000000000000.0, "iloveweiguangniang");

        System.out.println("下面是调用getAccountInfo()方法的输出:");
        System.out.println(b.getAccountInfo("iloveweiguangniang"));
        System.out.println("-----");

        System.out.println("下面是调用getBalance()方法的输出：（我的财富只能用科学计数法
ww）");
        System.out.println(b.getBalance("iloveweiguangniang"));
        System.out.println("-----");

        System.out.println("下面进行一次转账1314元: ");
        BankAccount bb = new BankAccount("12345678", "微光娘", 100.0,
"iloveEllery");
        System.out.println("转账前: \n" + "我的账户: \n"+
b.getAccountInfo("iloveweiguangniang") + "\n" + "微光娘的账户: \n"+
bb.getAccountInfo("iloveEllery"));
```

```
//转账方法返回boolean值，这里用if语句判断
if(b.transfer(bb, 1314.0, "iloveweiuguangniang")) {
    System.out.println("转账成功! ");
}
else {
    System.out.println("转账失败! ");
}
System.out.println("转账后: \n" +"我的账户: \n"+
b.getAccountInfo("iloveweiuguangniang") + "\n" +"微光娘的账户: \n"+
bb.getAccountInfo("iloveEllery"));
System.out.println("-----");
System.out.println("其他方法同理了，篇幅有限~");
}
}
```

下面是调用getAccountInfo()方法的输出：

卡号:****9810

账户持有人: Ellery

余额: 1.0E12

下面是调用getBalance()方法的输出：（我的财富只能用科学计数法ww
1.0E12

下面进行一次转账1314元：

转账前：

我的账户：

卡号:****9810

账户持有人: Ellery

余额: 1.0E12

微光娘的账户：

卡号:****5678

账户持有人: 微光娘

余额: 100.0

转账成功！

转账后：

我的账户：

卡号:****9810

账户持有人: Ellery

余额: 9.99999998686E11

微光娘的账户：

卡号:****5678

账户持有人: 微光娘

余额: 1414.0

其他方法同理了，篇幅有限~

最后的最后

以下内容是我写全部内容才写上的，看看我之前写的东西啥不足吧

- 几何图形那里，我每一个几何图形的构造方法里都没有写上关于写入数据的判断，我在TASK5前面还说了下这个非法数据的问题，不过当时确实没有意识到
- 依旧几何图形，那个三角形类太垃圾了(
参数写的莫名奇妙，有a,b,c,h四个变量，意义不明，h还不知道是哪个底边的高
确实应该减少变量，用海伦公式来算面积，还有在构造方法加上三角形三边关系这个数据判断
- 银行账户里面的问题很多上面已经提到过了，这里因为是复盘所以再说
 - 转账方法偷懒直接调用了存款方法，让转账时还需要收款方密码
 - 关于校验金额和校验密码两个方法，可能学长题目给出来的本意并不是想让我在其他方法调用它们去实现校验密码功能，可能我多写点判断好点
 - getBalance里那个密码错误的时候的else后面，我写了 `return -1`，这个是IEDA帮我补上的，因为我实在不知道该return啥了，其实也有点意义不明
 - 存款方法我没有判断数据合法性，可以存负数进去，越存越少
 - 卡号那里我用了substring，首先是我没有获取卡号长度，星号的个数不对
其次依旧是数据合法性，卡号可能太短了还没有四位，这时候会报错
- 还有一个总体上的问题
我写if-else的时候，一般来说是不论是否单行都加上大括号的
可能是我还没有适应，IDEA那个提示真的很烦，喜欢出来挡代码（
这时候大括号乱七八糟，最后我有些地方索性省略不写了，以后还是写上吧